Polynomial conversion of noncommutative formulas into abecedarian formulas with logarithmically many buckets

Prerona Chatterjee

doi:10.4230/LIPIcs.CCC.2021.7; CC BY 4.0

Editorial note. Editable body faithfully transcribes the complete formally published CC BY Sections1–6. Nonexclusive native archive and post-end-document old ideas remain private; excluded ranges/comments are blanked while retaining original locator lines. Formal global theorem numbering restored; unused font/git/todo packages omitted and bibliography venue-format macros expanded as ordinary text. Author mathematical statements and proofs unchanged. All production and validation on Yupeng only. Unused malformed note environment omitted from wrapper. Provide missing section autoref names before original renewcommands to accommodate assembly package order. This selection includes complete Sections1–4.3; introductory mentions of omitted Sections5/6 are rendered as the accurate original chapter numbers, without fabricated local labels. Explicitly restore original hyperref dependency for all author autoref commands.

Allowed standard prerequisites (editorial).

- Noncommutative polynomials, arithmetic formulas and ABPs; cited homogeneous multilinear formula lower bound. Original Sections1–2 and Theorem26 cite the standard lower bound; all source-local depth reduction, homogenization and degree amplification arguments included

1 Introduction

Algebraic Circuit Complexity is the study of multivariate polynomials and their classification based on how hard it is to compute them, using various computational models. The most well studied model is that of algebraic circuits. These are directed acyclic graphs that use algebraic operations like addition and multiplication over some field or ring, to compute polynomials. When the underlying graph is only allowed to be a tree, the model is that of algebraic formulas.

The central question in this area is whether the class VNP (algebraic analogue of the class NP) is contained in the class VP (algebraic analogue of the class P). Valiant [Val79] had shown that the permanent polynomial is complete for VNP. Thus, the VP vs VNP question essentially boils down to asking whether the $n \times n$ permanent can be computed by a $\mathrm{poly}(n)$-sized algebraic circuit.

In this paper, we are interested in polynomials that come from the non-commutative polynomial ring $\mathbb{F}\langle x_1, \ldots, x_n \rangle$, where the indeterminates do not commute with each other (that is, $xy \neq yx$ for indeterminates x, y). As a consequence, any monomial in a non-commutative polynomial $f \in \mathbb{F}\langle x_1, \ldots, x_n \rangle$ is essentially a string over the alphabet $\{x_1, \ldots, x_n\}$. This is a natural restriction and there has been a long line of work that studies non-commutative computation beginning with[1] the seminal work of Nisan [Nis91].

It was shown by Hrubeš, Wigderson and Yehudayoff [HWY10] that the non-commutative permanent polynomial is complete for the class $\mathsf{VNP}_{\mathrm{nc}}$ (the non-commutative version of VNP). Later Arvind, Joglekar and Raja [AJR16] gave a natural polynomial that is complete for the class of n-variate non-commutative polynomials computable by poly(n)-sized circuits (denoted by $\mathsf{VP}_{\mathrm{nc}}$).

The question of whether the classes $\mathsf{VP}_{\mathrm{nc}}$ and $\mathsf{VNP}_{\mathrm{nc}}$ are different is the central open problem in the non-commuatative setting. Although the general question of showing lower bounds against non-commutative circuits remains open, there has been significant progress in restricted settings [LMS16, LMP19, LLS19, ST18, FLOS20].

With respect to the general question, Hrubeš, Wigderson and Yehudayoff [HWY11] showed that a sufficiently strong super-linear lower bound for the classical sum-of-squares problem implies a separation between $\mathsf{VP}_{\mathrm{nc}}$ and $\mathsf{VNP}_{\mathrm{nc}}$. In another related work, Carmosino, Impagliazzo, Lovett and Mihajlin [CILM18] showed that proving mildly super-linear lower bounds against non-commutative circuits would imply exponential lower bounds against the same model.

One motivation for studying non-commutative computation is that it is possibly easier to prove strong lower bounds in this setting as compared to the usual commutative setting. At least intuitively, it seems harder to *cancel* monomials once they have been calculated when commutativity is not allowed amongst the variables. For example, the $n \times n$ determinant can be computed by an $O(n^3)$ algebraic circuit, but to the best of our knowledge there is no circuit for the non-commutative determinant of size $2^{o(n)}$. In fact, it was shown by Arvind and Srinivasan [AS18] that if the non-commutative determinant had a poly-sized circuit, then $\mathsf{VP}_{\mathrm{nc}} = \mathsf{VNP}_{\mathrm{nc}}$.

Even though a super-polynomial lower bound is not known for the non-commutative determinant against circuits, Nisan [Nis91] gave an exponential lower bound on the number of gates in any formula computing it. In contrast, the best lower bound known against formulas in the commutative setting is quadratic (for the elementary symmetric polynomial) [Nec66, Kal85, CKSV19].

A point to note about the lower bound given by Nisan however, is that the proof actually works for a computational model, called Algebraic Branching Programs (or ABPs), that is believed to be more general than algebraic formulas. In fact, Nisan [Nis91] gave an exact characterisation for the size of any ABP computing a non-commutative polynomial. As far as we are aware, any lower bound known against general non-commutative formulas uses this characterisation and hence is essentially a lower bound against non-commutative ABPs itself.

The motivating question for this work is whether there is a separation between the powers of ABPs and formulas in the non-commutative setting. Let

[1] Hyafil [Hya77] had considered non-commutative computation before this, but the main result in that paper is unfortuantely false as shown in [Nis91]

us denote the class of non-commutative polynomials over n variables that can be computed by poly(n)-sized ABPs by $\mathsf{VBP}_{\mathrm{nc}}$. Similarly, let $\mathsf{VF}_{\mathrm{nc}}$ denote the class of non-commutative polynomials over n variables that can be computed by poly(n)-sized formulas. The question is essentially whether $\mathsf{VBP}_{\mathrm{nc}}$ is contained in $\mathsf{VF}_{\mathrm{nc}}$.

This question had been posed by Nisan [Nis91], and the only work we are aware of that has made some progress with respect to this question is the one by Lagarde, Limaye and Srinivasan [LLS19]. They show that certain syntactically restricted non-commutative formulas (called Unique Parse Tree formulas) can not compute $\mathrm{IMM}_{n,n}$ unless they have size $n^{\Omega(\log n)}$.

In this paper, we study restrictions of a different kind. From here on, we will only be talking about non-commutative computation unless specifically mentioned otherwise.

1.1 Abecedarian Polynomials and Models That Compute Them

In [HWY11], Hrubeš *et al.* had defined the notion of *ordered* polynomials. A homogeneous polynomial of degree d is said to be ordered if the set of variables it depends on can be partitioned into d buckets such that variables occuring in position k only come from the k-th bucket. We generalise this notion by making the bucket indices *position independent*.

In English, an abecedarian word is one in which all of its letters are arranged in alphabetical order [MW19]. Since monomials are essentially words in the non-commutative setting, we call a polynomial abecedarian whenever the variables in all of its monomials are *arranged alphabetically*.

We call $(X_1, \ldots, X_m)$ (or $\{X_1, \ldots, X_m\}$, since the ordering is clear from context) a bucketing system of size m, for the set of variables $\{x_1, \ldots, x_n\}$, if it forms an *ordered* partition of $\{x_1, \ldots, x_n\}$. A non-commutative polynomial $f \in \mathbb{F}\langle x_1, \ldots, x_n\rangle$ is said to be abecedarian with respect to the bucketing system $\{X_1, \ldots, X_m\}$, if

- $\{X_1, \ldots, X_m\}$ is a bucketing system for $\{x_1, \ldots x_n\}$;
- the variables in each of the monomials in f are arranged in *non-decreasing* order of their bucket indices.

A formal definition can be found in Section 2.1.

"Getting our Hands Dirty" with Abecedarian Polynomials

Before moving ahead, let us take a look at an example of an abecedarian polynomial. Given a commutative polynomial $f \in \mathbb{F}[x_1, \ldots, x_n]$, define its non-commutative analogue, $f^{(\mathrm{nc})}$ as follows.

> f and $f^{(\mathrm{nc})}$ look essentially the same, except that variables in every monomial in $f^{(\mathrm{nc})}$ are arranged in non-decreasing order of their indices.

Then, $f^{(\mathrm{nc})}$ is abecedarian with respect to the bucketing system $\{X_i \ : \ X_i = \{x_i\}\}$.

Let us also look at a possibly important polynomial that is *not* abecedarian with respect to the bucketing system $\{X_i \ : \ X_i = \{x_i\}\}$. Consider the *arc-full*

rank polynomial[2] f, but as a non-commutative polynomial, f', in the following sense.

> Let $\mathcal{A}$ be the ABP that computes f and think of that as a non-commutative ABP $\mathcal{A}'$. Then, f' is the polynomial computed by $\mathcal{A}'$.

It is not hard to see that across different monomials in f', the order in which variables are arranged is not consistent. Thus, f' is not abecedarian with respect to the given bucketing system.

A final point to note before we move ahaed is that a polynomial might be abecedarian with respect to different bucketing systems[3]. In fact, even the sizes of the different bucketing systems might be different. For example, the polynomial

$$\mathrm{ESYM}_{n,d}^{(\mathsf{ord})} = \sum_{1 \le i_1 < \ldots < i_d \le n} x_{i_1}^{(1)} \cdots x_{i_d}^{(d)}$$

is abecedarian with respect to the bucketing system $\left\{X_k = \left\{x_i^{(k)} \ : \ i \in [n]\right\}\right\}$ which has size d, as well as $\left\{X_i = \left\{x_i^{(k)} \ : \ k \in [d]\right\}\right\}$ which has size n.

Abecedarian Models of Computation

Hrubeš *et al.* [HWY11] had defined *ordered circuits*, a model naturally suited to compute ordered polynomials. We generalise this notion to define circuits that naturally compute abecedarian polynomials. Going one step further, we also define abecedarian ABPs and abecedarian formulas.

Suppose f is an abecedarian polynomial with respect to the bucketing system $\{X_1, \ldots, X_m\}$. For any $1 \le a \le b \le m+1$, $f[a,b)$ is a sub-polynomial of f defined as follows.

- For any $a \in [m+1]$, $f[a,a)$ is the constant term in f.
- For $1 \le a < b \le m+1$, $f[a,b)$ contains only those monomials of f in which the first variable is from bucket X_a and the last variable is from any of the buckets in the set $\{X_a, \ldots, X_{b-1}\}$.

A circuit is said to be abecedarian if every gate v in it can be labelled by a tuple (a,b) such that if f_v is the polynomial computed at that gate, then $f_v = f_v[a,b)$. We call a formula abecedarian if it has a similar syntactic property at every gate. For formal definitions, see Definition 18 and Definition 20 respectively. On the other hand, an ABP is said to be abecedarian when every vertex in it can be labelled by a bucket index such that if f is the polynomial computed between vertices labelled with indices a and b respectively, then $f = f[a, b+1)$. Definition 19 is a formal definition.

1.2 Our Main Results

Our main result is a super-polynomial separation between abecedarian formulas and ABPs.

[2]The hard polynomial constructed by Dvir, Malod, Perifel and Yehudayoff [DMPY12] to separate the powers of formulas and ABPs in the multilinear setting.

[3]Every polynomial $f \in \mathbb{F}\langle x_1, \ldots, x_n \rangle$ is abecedarian with respect to the bucketing system $\{X\}$ for $X = \{x_1, \ldots, x_n\}$.

Theorem 1 (Separating Abecedarian Formulas and Abecedarian ABPs)**.** *Define*

$$\mathsf{linked_CHSYM}_{n,d}(\mathbf{x}) = \sum_{i_0=1}^{n} \left(\sum_{i_0 \le i_1 \le \ldots \le i_d \le n} x_{i_0,i_1} \cdot x_{i_1,i_2} \cdots x_{i_{d-1},i_d} \right)$$

to be the linked *complete homogeneous polynomial over n-variables of degree d.*

Note that $\mathsf{linked_CHSYM}_{n,d}(\mathbf{x})$ *is* abecedarian *with respect to the bucketing system $\{X_i : i \in [n]\}$ where $X_i = \{x_{i,j} : i \le j \le n\}$. With respect to this bucketing system,*

1. $\mathsf{linked_CHSYM}_{n,d}(\mathbf{x})$ *has an* abecedarian *ABP of size $O(nd)$;*
2. *any* abecedarian *formula computing* $\mathsf{linked_CHSYM}_{n/2,\log n}(\mathbf{x})$ *has size* $n^{\Omega(\log \log n)}$.

That is, there is a super-polynomial separation between abecedarian *ABPs and* abecedarian *formulas.*

Our second main result shows that in certain settings, formulas computing abecedarian polynomials can be assumed to be abecedarian without loss of generality.

Theorem 2 (Converting Formulas into Abecedarian Formulas)**.** *Let f be an* abecedarian *polynomial with respect to a bucketing system of size m, and $\mathcal{F}$ be a formula of size s computing f. If $m = O(\log s)$, then there is an* abecedarian *formula $\mathcal{F}'$ computing f of size* $\mathrm{poly}(s)$.

In other words, an $n^{\omega(1)}$ lower bound against abecedarian formulas computing any polynomial that is abecedarian with respect to a bucketing system of size $O(\log n)$, would result in a super-polynomial lower bound against general non-commutative formulas.

These statements suggest a new approach towards resolving the general $\mathsf{VF}_{\mathrm{nc}}$ vs $\mathsf{VBP}_{\mathrm{nc}}$ question.

Connections to the General $\mathsf{VF}_{\mathrm{nc}}$ vs $\mathsf{VBP}_{\mathrm{nc}}$ Question

Theorem 1 gives a separation between abecedarian formulas and ABPs. On the other hand, Theorem 2 shows that if we are given a formula that computes a polynomial that is abecedarian with respect to a bucketing system of *small* size, then we can assume that the formula is abecedarian without loss of generality. Unfortunately, the bucketing system with respect to which our *hard polynomial* from Theorem 1 is abecedarian, is *not small* in size. Thus, the general question of whether $\mathsf{VBP}_{\mathrm{nc}}$ is contained in $\mathsf{VF}_{\mathrm{nc}}$ or not still remains open.

However, there are two natural questions that arise at this point.

1. Can any formula computing an abecedarian polynomial be converted to an abecedarian formula without much blow-up in size, irrespective of the size of the bucketing system?

2. Is there a polynomial f which is abecedarian with respect to a bucketing system that has *small* size such that f witnesses a separation between abecedarian formulas and ABPs?

Clearly, a positive answer to either of these questions would imply that $\mathsf{VBP}_{\mathrm{nc}} \neq \mathsf{VF}_{\mathrm{nc}}$. In particular, a super-polynomial lower bound against abecedarian formulas for a polynomial very similar to the one we used to show our separation would separate $\mathsf{VBP}_{\mathrm{nc}}$ and $\mathsf{VF}_{\mathrm{nc}}$.

Corollary 3. *Let the polynomial* linked_$\mathrm{CHSYM}_{n,d}(\mathbf{x})$ *be as defined in Theorem 1. An* $n^{\omega(1)}$ *lower bound against* abecedarian *formulas for* linked_$\mathrm{CHSYM}_{\log n,n}(\mathbf{x})$ *would imply a super-polynomial separation between non-commutative ABPs and formulas.*

In fact our proof technique also shows that a super-polynomial lower bound against *homogeneous* formulas for our hard polynomial would separate $\mathsf{VBP}_{\mathrm{nc}}$ and $\mathsf{VF}_{\mathrm{nc}}$.

Corollary 4. *Let* linked_$\mathrm{CHSYM}_{n,d}$ *be as defined in Theorem 1. An* $n^{\omega(1)}$ *lower bound against homogeneous formulas for* linked_$\mathrm{CHSYM}_{n,\log n}(\mathbf{x})$ *would result in a super-polynomial separation between ABPs and formulas in the non-commutative setting.*

1.3 Proof Overview

We now give a proof overview of our main theorems.

Separating Abecedarian Formulas and ABPs

Let us first consider Theorem 1.

Theorem 1 (Separating Abecedarian Formulas and Abecedarian ABPs)**.** *Define*

$$\text{linked_}\mathrm{CHSYM}_{n,d}(\mathbf{x}) = \sum_{i_0=1}^{n} \left(\sum_{i_0 \leq i_1 \leq \ldots \leq i_d \leq n} x_{i_0,i_1} \cdot x_{i_1,i_2} \cdots x_{i_{d-1},i_d} \right)$$

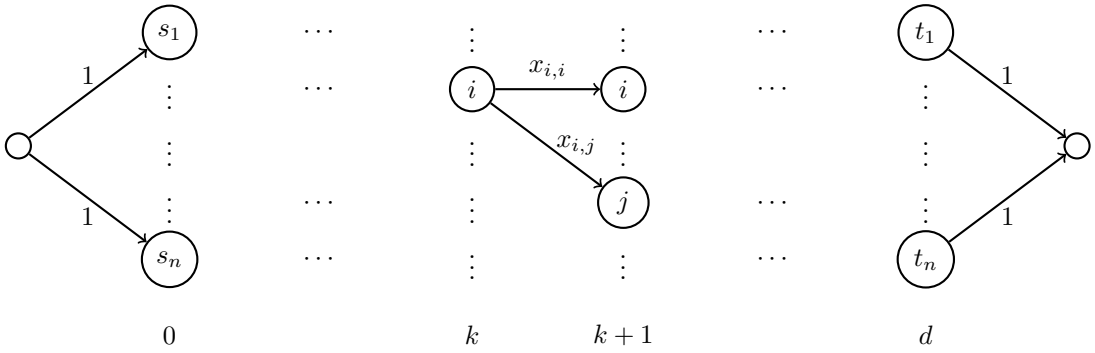
to be the linked *complete homogeneous polynomial over* n*-variables of degree* d*.*

Note that linked_$\mathrm{CHSYM}_{n,d}(\mathbf{x})$ *is* abecedarian *with respect to the bucketing system* $\{X_i : i \in [n]\}$ *where* $X_i = \{x_{i,j} : i \leq j \leq n\}$*. With respect to this bucketing system,*

1. linked_$\mathrm{CHSYM}_{n,d}(\mathbf{x})$ *has an* abecedarian *ABP of size* $O(nd)$*;*
2. *any* abecedarian *formula computing* linked_$\mathrm{CHSYM}_{n/2,\log n}(\mathbf{x})$ *has size* $n^{\Omega(\log \log n)}$*.*

That is, there is a super-polynomial separation between abecedarian *ABPs and* abecedarian *formulas.*

A *small* abecedarian ABP computing linked_$\mathrm{CHSYM}_{n,d}(\mathbf{x})$ is essentially the following.



For the lower bound, assume that we have been given a *small* abecedarian formula computing the polynomial. We then keep modifying this formula till we get a *small* homogeneous multilinear formula computing the *elementary symmetric polynomial* of degree $n/2$. We then use the known lower bound against homogeneous multilinear formulas for this polynomial (shown by Hrubeš and Yehudayoff [HY11]), to get a contradiction.

Let us spell out the proof in some more detail.

Step 1: Assume that we are given an abecedarian formula computing linked_$\mathrm{CHSYM}_{n/2,\log n}(\mathbf{x})$ of size $O(n^{\varepsilon \log\log n})$. Since the degree of the polynomial being computed is *small*, we can assume that there is in fact a *homogeneous* abecedarian formula computing linked_$\mathrm{CHSYM}_{n/2,\log n}(\mathbf{x})$ of size $O(n^{c\cdot\varepsilon \log\log n})$ for some constant c independent of ε.

Step 2: Using the homogeneous abecedarian formula from Step 1, we obtain a *structured* homogeneous abecedarian formula, of size $O(n^{c\cdot\varepsilon \log\log n})$, computing linked_$\mathrm{CHSYM}_{n/2,\log n}(\mathbf{x})$.

Step 3: We consider the complete homogeneous polynomial over n variables of degeree d

$$\mathrm{CHSYM}_{n,d}(\mathbf{x}) = \sum_{1\le i_1\le \ldots \le i_d\le n} x_{i_1}\cdots x_{i_d},$$

and show that there is a homogeneous abecedarian formula of size $\mathrm{poly}(n)$ that computes $\mathrm{CHSYM}_{n/2,\log n}(\mathbf{x})$.

Step 4: If the formula in Step 2 has size s and that in Step 3 has size s', then we show that there is a homogeneous abecedarian formula of size $(s\cdot s')$ computing $\mathrm{CHSYM}_{n/2,\log^2 n}(\mathbf{x})$.

Step 5: Next, we show that Step 4 can be used repeatedly at most $O(\log n/\log\log n)$ times, to obtain a homogeneous abecedarian formula computing $\mathrm{CHSYM}_{n/2,n/2}(\mathbf{x})$ of size $O(n^{c\cdot\varepsilon\log n})$.

Step 6: Using the formula obtained in Step 5, we get a homogeneous multilinear formula computing the elementary symmetric polynomial of degree $n/2$, of size $O(n^{c\cdot\varepsilon\log n})$.

Step 7: Finally, we choose ε in such a way that Step 6 contradicts the theorem in [HY11].

The crucial observation that makes this proof work, is that the polynomial we are working with is structured enough for us to be able to amplify its degree in a systematic way (without blowing up the size by much). This is the 4th step in the description above.

Apart from that, the entire proof essentially boils down to the fact that when formulas are computing low degree polynomials, there are some additional tricks available to make them more structured. A complete proof of Theorem 1 can be found in Section 5 of the original publication.

We now elaborate a little on the first step, since the observations made to prove this step are quite general and possibly useful in various settings. These statements are known to be true in the commutative setting and their proofs in the non-commutative setting are fairly similar to the ones for their commutative counterparts. We state them here nevertheless, since to the best of our knowledge, they have not been stated formally before for the non-commutative setting.

Homogenising Non-Commutative Formulas computing Low Degree Polynomials Raz [Raz13] had shown that if there is a formula computing a homogeneous polynomial of *low* degree in the commutative world, then it can be assumed without loss of generality that the formula is homogeneous. We show that this statement is true even in the non-commutative setting.

Lemma 5 (Homogenising Non-Commutative Formulas computing Low Degree Polynomials)**.** *Suppose f is a non-commutative homogeneous polynomial of degree $d = O(\log s)$, where s is its ABP complexity. Also, suppose $\mathcal{F}$ is a fan-in 2 formula of size s' that computes f. Then there is a homogeneous formula $\mathcal{F}'$ computing f, that has size $\text{poly}(s')$ and whose multiplication gates have fan-in 2.*

Further, if $\mathcal{F}$ was abecedarian *with respect to some bucketing system, then $\mathcal{F}'$ is also* abecedarian *with respect to the same bucketing system.*

The only thing that needs to be checked for Raz's proof to work in this setting is whether non-commutative formulas can be depth-reduced to log-depth. We show that infact they can be.

Depth Reduction for Non-Commutative Formulas Brent [Bre74] had shown that if there is a formula of size s computing a commutative polynomial f, then there is a formula of depth $O(\log s)$ and size $\text{poly}(s)$ that computes the same polynomial. We show that this is also true in the non-commutative setting. The proof is almost exactly along the same lines as the one by Brent [Bre74], just analysed carefully.

Lemma 6 (Depth Reduction of Non-Commutative Formulas)**.** *If there is a fan-in 2 formula $\mathcal{F}$ of size s computing a non-commutative polynomial f, then there is a fan-in 2 formula $\mathcal{F}'$ of size $\text{poly}(s)$ and depth $O(\log(s))$ computing f.*

Further if $\mathcal{F}$ is homogeneous, $\mathcal{F}'$ is also homogeneous. Similarly, if $\mathcal{F}$ is abecedarian *with respect to some bucketing system, then $\mathcal{F}'$ is also* abecedarian *with respect to the same bucketing system.*

The statement is slightly surprising because Nisan [Nis91] had shown that such a statement is false for non-commutative circuits[4]. Lemma 6 says that this

[4]Unlike commutative circuits [VSBR83].

is not the case with formulas.

This observation is not only crucial for proving some of our statements, it also answers a question posed by Nisan [Nis91]. For a polynomial f, let $D(f)$ denote the minimum depth among all fan-in 2 circuits computing f, $B(f)$ the size of the smallest ABP computing f, and $F(f)$ the size of the smallest fan-in 2 formula computing f. Nisan [Nis91] had shown that if f has degree d, then

$$D(f) \leq O(\log B(f) \cdot \log d) \quad \text{and} \quad F(f) \leq 2^{D(f)}.$$

He had then asked whether $D(f) \leq O(\log F(f))$ or not. Lemma 6 clearly implies that, in particular, the answer to this question is YES.

Converting Formulas into Abecedarian Formulas

Next we go over the proof idea of Theorem 2.

Theorem 2 (Converting Formulas into Abecedarian Formulas)**.** *Let f be an* abecedarian *polynomial with respect to a bucketing system of size m, and $\mathcal{F}$ be a formula of size s computing f. If $m = O(\log s)$, then there is an* abecedarian *formula $\mathcal{F}'$ computing f of size* $\text{poly}(s)$.

We prove this statement by first converting the given formula $\mathcal{F}$ into an abecedarian circuit $\mathcal{C}$, and then unravelling it to get an abecedarian formula $\mathcal{F}'$ computing the same polynomial.

The first step is fairly straightforward. The proof is along the same lines as that for homogenising circuits. The only difference is that we keep track of bucket indices of the variables on either ends of the monomials being computed at the gates, instead of their degrees.

In the second step, we convert $\mathcal{C}$ into a formula $\mathcal{F}'$. In order to do that, we need to recompute vertices every time it is reused. Thus, to give an upper bound on the size of $\mathcal{F}'$, we need to find an upper bound on the number of distinct paths from any vertex in $\mathcal{C}$ to the root. This analysis is done in a way similar to the one by Raz [Raz13] in his proof of the fact that formulas computing low degree polynomials can be homogenised without much blow-up in size. The requirement of the size of the bucketing system being small also arises because of this analysis.

The only additional point that needs to be checked for the proof to go through is that similar to the commutative setting, non-commutative formulas can be depth reduced as well (Lemma 6). A complete proof of Theorem 2 can be found in Section 4.3.

1.4 Other Results: A Complete View of the Abecedarian World

We now go over some other results that helps in completing the view of the abecedarian world.

As mentioned earlier, Hrubeš *et al.* [HWY11] had defined ordered circuits, a model naturally suited to compute ordered polynomials. They had then gone on to show that without loss of generality, any circuit computing an ordered polynomial can be assumed to be ordered[5]. We show that even in the abecedarian setting, such a statement is true.

[5] Theorem 7.1 in [HWY11].

Observation 7 (Converting Circuits into Abecedarian Circuits)**.** *Let f be an* abecedarian *polynomial with respect to a bucketing system of size m, and $\mathcal{C}$ be a circuit of size s computing f. Then there is an* abecedarian *circuit $\mathcal{C}'$ computing f of size $O(m^3 s)$.*

What this implies is that an $n^{\omega(1)}$ lower bound against abecedarian circuits for any explicit polynomial that is abecedarian would result in a super-polynomial lower bound against general non-commutative circuits. We also show that an analogous statement is true even for abecedarian ABPs.

Observation 8 (Converting ABPs into Abecedarian ABPs)**.** *Suppose f is an* abecedarian *polynomial with respect to a bucketing system of size m. If there is an ABP $\mathcal{A}$ of size s computing it, then there is an* abecedarian *ABP $\mathcal{A}'$ computing it of size $O(ms)$.*

Next, let us define various classes of abecedarian polynomials.

Let abc - $\mathsf{VP}_{\mathrm{nc}}$ denote the class of abecedarian polynomials that can be computed by poly-sized abecedarian circuits. Similarly let abc - $\mathsf{VBP}_{\mathrm{nc}}$ and abc - $\mathsf{VF}_{\mathrm{nc}}$ denote the classes of abecedarian polynomials that can be computed by poly-sized abecedarian ABPs and abecedarian formulas respectively. We first note that the logical inclusions that should hold, do hold.

Observation 9 (The Usual Inclusions)**.** *Let* abc - VP_{nc}*,* abc - VBP_{nc} *and* abc - VF_{nc} *denote the classes of* abecedarian *polynomials over n variables that can be computed by* $\mathrm{poly}(n)$ *sized* abecedarian *circuits,* abecedarian *ABPs and* abecedarian *formulas respectively. Then,*

$$\text{abc - } \mathsf{VF}_{nc} \subseteq \text{abc - } \mathsf{VBP}_{nc} \subseteq \text{abc - } \mathsf{VP}_{nc}.$$

We also observe that if a degree d polynomial has an abecedarian ABP of size s, then it has an abecedarian formula of size $O(s^{\log d})$ via the usual divide-and-conquer algorithm.

Observation 10 (Converting Abecedarian ABPs into Abecedarian Formulas)**.** *Suppose f is an* abecedarian *polynomial of degree d. If there is an* abecedarian *ABP $\mathcal{A}$ of size s computing it, then there is an* abecedarian *formula $\mathcal{F}$ computing f of size $O(s^{\log d})$.*

What Theorem 1 essentially shows is that the blow-up observed in Observation 10 is tight. Finally, it is not hard to see that Nisan's proof can be modified to give an exponential separation between abecedarian ABPs and abecedarian circuits.

General Formula Lower Bound from Homogeneous Formula Lower Bound

We end by showing that homogeneous formula lower bounds for some well-studied polynomials would lead to separating $\mathsf{VF}_{\mathrm{nc}}$ and $\mathsf{VBP}_{\mathrm{nc}}$. These statements are corollaries of Lemma 5.

Corollary 11. *A $2^{\omega(n)}$ lower bound against homogeneous formulas computing the $n \times n$ determinant polynomial,* $\mathrm{Det}_n(\mathbf{x})$*, implies* $\mathsf{VF}_{nc} \neq \mathsf{VBP}_{nc}$.

Remark 12. *Nisan [Nis91] had shown that the ABP complexity of* $\mathrm{Det}_n(\mathbf{x})$ *and* $\mathrm{Perm}_n(\mathbf{x})$ *is* $2^{\Theta(n)}$*. In the case of* $\mathrm{Perm}_n(\mathbf{x})$*, we also have a* $2^{O(n)}$ *upper bound due to Ryser [Rys63]. However, to the best of our knowledge, there is no formula computing* $\mathrm{Det}_n(\mathbf{x})$ *of size* $n^{o(n)}$*. Hence there is a possibility that a* $2^{\omega(n)}$ *lower bound can be shown against formulas for this polynomial.* ◊

Corollary 13. *An* $n^{\omega(1)}$ *lower bound against homogeneous formulas computing the* n*-variate iterated matrix multiplication polynomial of degree* $\log n$*,* $\mathrm{IMM}_{n,\log n}(\mathbf{x})$*, implies a super-polynomial separation between ABPs and formulas in the non-commutative setting.*

To put the requirement of degree being $O(\log n)$ in perspective, note the following.

Remark 14 (Analogous to Remark 5.12 in [LLS19])**.** *The standard divide and conquer approach for computing the iterated matrix multiplication polynomial* $\mathrm{IMM}_{n,d}$ *yields a (homogeneous) formula of size* $n^{O(\log d)}$*. It would be quite surprising if this standard algorithm were not optimal in terms of formula size.*

Intuitively, improving on the standard divide and conquer algorithm gets harder as d *gets smaller. This is because any (homogeneous) formula of size* $n^{o(\log d)}$ *for computing* $\mathrm{IMM}_{n,d}$ *can be used in a straightforward manner to recursively obtain (homogeneous) formulas for* $\mathrm{IMM}_{,n,D}$ *of size* $n^{o(logD)}$ *for any* $D > d$*. The case of smaller* d*, which seems harder algorithmically, is thus a natural first candidate for lower bounds.* ◊

1.5 Structure of the Paper

We begin, in Section 2, with formal definitions for abecedarian polynomials and naturally restricted version of circuits, ABPs and formulas that compute them. Then, in Section 3, we prove some structural statements, namely Lemma 6 and Lemma 5. In Section 4, we prove Theorem 2 along with Observation 7 and Observation 8. We then prove our main result (Theorem 1), that gives a super-polynomial separation between abecedarian formulas and ABPs, in Section 5 of the original publication. Finally, in Section 6 of the original publication, we prove the remaining statements mentioned above.

2 Preliminaries

Let us begin by formally defining abecedarian polynomials and the naturally restricted versions of circuits, ABPs and formulas that compute them.

2.1 Abecedarian Polynomials

First, we define the notion of a bucketing system.

Definition 15 (Bucketing System for a Set of Indeterminates)**.** *Suppose* $\{x_1, \ldots, x_n\}$ *is a set of indeterminates. An ordered set* $(X_1, \ldots, X_m)$ *is said to be a bucketing system for it, if* $\{X_1, \ldots, X_m\}$ *forms a partition of the set* $\{x_1, \ldots, x_n\}$*.*

In this case, we may also denote the bucketing system $(X_1, \ldots, X_m)$ *by* $\{X_1, \ldots, X_m\}$*, since the ordering among* $X_1, \ldots X_m$ *is clear from context.* ◊

Next, let us formally define abecedarian polynomials.

Definition 16 (Abecedarian Polynomials)**.** *A polynomial $f \in \mathbb{F}\langle x_1, \ldots, x_n\rangle$ of degree d is said to be* abecedarian *with respect to a bucketing system $\{X_1, \ldots, X_m\}$ for $\{x_1, \ldots x_n\}$, if*

$$f = f[\emptyset) + \sum_{k=1}^{d} \left(\sum_{1 \le i_1 \le \cdots \le i_k \le m} f[X_{i_1}, \ldots, X_{i_k}] \right)$$

where $f[\emptyset)$ is the constant term in f, and for any $k \in [d]$, $f[X_{i_1}, \ldots, X_{i_k}]$ is defined as follows. For a polynomial f, $f[X_{i_1}, \ldots, X_{i_k}]$ is the homogeneous polynomial of degree k such that for every monomial α,

$$\mathsf{coeff}_\alpha(f[X_{i_1}, \ldots, X_{i_k}]) = \begin{cases} \mathsf{coeff}_\alpha(f) & \text{if } \alpha = x_{\ell_1} \cdots x_{\ell_k} \text{ with } x_{\ell_j} \in X_{i_j} \text{ for every } j \in [k] \\ 0 & \text{otherwise.} \end{cases}$$

In this case, we say that f is abecedarian *with respect to $\{X_1, \ldots, X_m\}$, a bucketing system of size m.* ♢

Abecedarian polynomials are essentially generalisations of ordered polynomials (defined by Hrubeš, Wigderson and Yehudayoff [HWY11]). A homogeneous polynomial, of degree d, is said to be ordered if the set of variables it depends on can be partitioned into d buckets such that variables occuring in position k only come from the k-th bucket.

It is easy to see that any ordered polynomial is also abecedarian with respect to the same bucketing system. This is because position indices are always increasing. For example, consider the following version of the *complete homogeneous symmetric polynomial*.

$$\mathrm{CHSYM}^{(\mathsf{ord})}_{n,d}(\mathbf{x}) = \sum_{1 \le i_1 \le \ldots \le i_d \le n} x^{(1)}_{i_1} \cdots x^{(d)}_{i_d}.$$

It is both ordered as well as abecedarian with respect to the buckets $\left\{X_k = \left\{x^{(k)}_i \ : \ i \in [n]\right\}\right\}$.

However, note that there are homogeneous polynomials that are abecedarian but not ordered. The following version of the same polynomial is an example.

$$\mathrm{CHSYM}_{n,d}(\mathbf{x}) = \sum_{1 \le i_1 \le \ldots \le i_d \le n} x_{i_1} \cdots x_{i_d}$$

is abecedarian with respect to $\{X_i \ : \ X_i = \{x_i\}\}$, but is not ordered.

The reason is that for a polynomial to be ordered, the bucket labels have to essentially be position labels. On the other hand, for a polynomial to be abecedarian with respect to a bucketing system, the bucket labels can be independent of position.

For example, note that $\mathrm{CHSYM}^{(\mathsf{ord})}_{n,d}(\mathbf{x})$ is abecedarian with respect to the bucketing system $\left\{X_i = \left\{x^{(k)}_i \ : \ k \in [d]\right\}\right\}$ along with the one mentioned earlier.

We now move on to defining algebraic models that naturally compute abecedarian polynomials.

2.2 Abecedarian Circuits

Homogeneous formulas have the property that any vertex can be labelled by a tuple of position indices (a, b) such that all the monomials being computed at that vertex occur exactly from position a to position b in the final polynomial that is being computed by it. Hrubeš *et al.* [HWY11] defined *ordered* circuits to be those circuits that have this property.

A circuit computing a degree d polynomial $f \in \mathbb{F}\langle x_1, \ldots, x_n \rangle$ is said to be ordered, if $\{X_1, \ldots, X_d\}$ forms a partition of $\{x_1, \ldots, x_n\}$ such that

- every gate v in the circuit is labelled by a tuple of position indices (a, b);
- if f_v is the polynomial computed at v, then
  - f_v is homogeneous and has degree $(b - a + 1)$;
  - every monomial in f_v is a product of exactly one variable from each of the buckets $X_a, \ldots, X_b$, multiplied in increasing order of their bucket indices.

We generalise this notion to define circuits that naturally compute abecedarian polynomials. Before we can do that, we need the notion of sub-polynomials of any abecedarian polynomial.

Definition 17 (Sub-Polynomials of an Abecedarian Polynomial)**.** *Suppose f is an* abecedarian *polynomial with respect to the bucketing system $\{X_1, \ldots, X_m\}$, and has degree d. For any $1 \leq a \leq b \leq m + 1$, $f[a, b)$ is the sub-polynomial of f defined as follows.*

- *For any $a \in [m + 1]$, $f[a, a) = f[\emptyset)$ is the constant term in f.*
- *For any $1 \leq a < b \leq m + 1$,*

$$f[a, b) = \sum_{k=1}^{d} \left(\sum_{\substack{i_1, \ldots, i_k \in [m] \\ a = i_1 \leq \cdots \leq i_k < b}} f[X_{i_1}, \ldots, X_{i_k}] \right)$$

where $f[X_{i_1}, \ldots, X_{i_k}]$ is as defined in Definition 16.

Further, we say that a polynomial f is of type *$[a, b)$ if $f = f[a, b)$.* ◊

Let us now formally define abecedarian circuits.

Definition 18 (Abecedarian Circuits)**.** *For any $a, b \in \mathbb{N}$, let $[a, b)$ denote a set of the form $I = \{i \ : \ a \leq i < b\}$. As a convention, $[a, a)$ denotes the empty set for every $a \in \mathbb{N}$.*

A multi-output circuit $\mathcal{C}$ is said to be abecedarian *when*

- *every gate v in $\mathcal{C}$ is associated with a set $I_v = [a, b)$;*
- *if f_v is the polynomial computed at v, then $f_v = f[a, b)$;*
- *if $v = v_1 + v_2$, then $I_v = I_{v_1} = I_{v_2}$;*
- *if $v = v_1 \times v_2$ with $I_v = [a, a)$, then $I_{v_1} = I_{v_2} = [a, a)$*

- *if $v = v_1 \times v_2$ with $I_v = [a, b)$ and $a < b$, then one of the following is true*
  - *$I_{v_1} = [a, b)$ and $I_{v_2} = [b, b)$;*
  - *$I_{v_1} = [a, a)$ and $I_{v_2} = [a, b)$;*
  - *there exists $a \leq c < b$ such that $I_{v_1} = [a, c + 1)$ and $I_{v_2} = [c, b)$.*

The polynomial computed by $\mathcal{C}$ is the sum of the polynomials computed at the different output gates. $\Diamond$

Next, we define abecedarian ABPs and abecedarian formulas as the restricted versions of ABPs and formulas respectively, that naturally compute abecedarian polynomials.

2.3 Abecedarian ABPs and Formulas

Homogeneous ABPs have the property that every vertex in it is labelled by a position index such that, polynomials computed between vertices labelled with indices a and b only contain monomials between positions a and $(b - 1)$.

We define abecedarian ABPs analogously except that the labels on the vertices are bucket labels instead of position labels. These restricted ABPs naturally compute abecedarian polynomials.

Definition 19 (Abecedarian ABPs)**.** *A multi-input, multi-output ABP $\mathcal{A}$ is said to be* abecedarian *when*

- *every vertex in it is labelled by a bucket index;*
- *if f is the polynomial computed between vertices labelled with indices a and b respectively, then $f = f[a, b + 1)$.*

The polynomial computed by $\mathcal{A}$ is the sum of all the polynomials computed between the various (input, output) gate pairs. $\Diamond$

Similarly, we define abecedarian formulas as analogues of homogeneous formulas, with the labels again referring to bucket indices instead of position indices.

Definition 20 (Abecedarian Formulas)**.** *Let sets of the form $[a, b)$, with $a, b \in \mathbb{N}$, be as defined in Definition 18. Suppose $\mathcal{F}$ is a formula computing a polynomial f that is* abecedarian *with respect to a bucketing system of size m. Then $\mathcal{F}$ is said to be* abecedarian *if $\mathcal{F}$ is in fact a collection of formulas $\{\mathcal{F}_i : i \in [m]\}$ such that for every $i \in [m]$,*

- *$\mathcal{F}_i$ computes the polynomial $f[i, m + 1)$;*
- *every gate v in $\mathcal{F}_i$ is associated with a set $I_v = [a, b)$, and in particular, the root node must be associated with the set $[i, m + 1)$*
- *if f_v is the polynomial computed at v, then $f_v = f_v[a, b)$;*
- *if $v = v_1 + v_2$, then $I_v = I_{v_1} = I_{v_2}$;*
- *if $v = v_1 \times v_2$ with $I_v = [a, a)$, then $I_{v_1} = I_{v_2} = [a, a)$*
- *if $v = v_1 \times v_2$ with $I_v = [a, b)$ and $a < b$, then one of the following is true*

- $I_{v_1} = [a, b)$ *and* $I_{v_2} = [b, b)$*;*
- $I_{v_1} = [a, a)$ *and* $I_{v_2} = [a, b)$*;*
- *there exists* $a \leq c < b$ *such that* $I_{v_1} = [a, c+1)$ *and* $I_{v_2} = [c, b)$*.*

The polynomial computed by $\mathcal{F}$ *is the sum of the polynomials computed by the various* $\mathcal{F}_i$*s. Further,* $\mathcal{F}$ *is said to be homogeneous if each* $\mathcal{F}_i$ *is homogeneous.* ◊

With these definitions in mind, we now move to proving some structural statements.

3 Structural Statements

In this section, we prove two structural statements in the non-commutative setting that are known to be true in the commutative setting. Apart from being crucial to our proofs, they are possibly interesting observations in their own right.

3.1 Depth Reduction for Non-Commutative Formulas

Brent [Bre74] had shown that if there is a formula of size s computing a commutative polynomial f, then there is a formula of depth $O(\log s)$ and size $\text{poly}(s)$ that computes the same polynomial. We show that this is also true in the non-commutative setting.

The proof is essentially the same as the one by Brent [Bre74], just analysed carefully. We give the complete proof for the sake of completeness.

Lemma 6 (Depth Reduction of Non-Commutative Formulas)**.** *If there is a fan-in* 2 *formula* $\mathcal{F}$ *of size* s *computing a non-commutative polynomial* f*, then there is a fan-in* 2 *formula* $\mathcal{F}'$ *of size* $\text{poly}(s)$ *and depth* $O(\log(s))$ *computing* f*.*

Further if $\mathcal{F}$ *is homogeneous,* $\mathcal{F}'$ *is also homogeneous. Similarly, if* $\mathcal{F}$ *is* abecedarian *with respect to some bucketing system, then* $\mathcal{F}'$ *is also* abecedarian *with respect to the same bucketing system.*

Proof. Suppose $\mathcal{F}$ is a fan-in 2 formula of size s that computes f. Then, we claim the following.

Claim 21. *Suppose* $\mathcal{F}_0$ *is a formula computing a polynomial* f_0 *and has fan-in 2. Then the there exist sub-formulas,* $L, \mathcal{F}_1, R, \mathcal{F}_2$*, of* $\mathcal{F}_0$ *such that*

- $\mathcal{F}'_0 = L \cdot \mathcal{F}_1 \cdot R + \mathcal{F}_2$ *also computes* f_0*;*
- *each of* $L, \mathcal{F}_1, R, \mathcal{F}_2$ *have size at least* $(s/3)$ *and at most* $(2s/3)$*;*
- *if* $\mathcal{F}_0$ *is homogeneous, then so are* $L, \mathcal{F}_1, R, \mathcal{F}_2$*;*
- *if* $\mathcal{F}_0$ *is* abecedarian *with respect to some bucketing system,* f_{left}*,* f_1*,* f_{right}*,* f_2 *are polynomials computed by* $L, \mathcal{F}_1, R, \mathcal{F}_2$ *respectively and* $f_0 = f_0[a, b)$*, then* $f_2 = f_2[a, b)$ *and*
  - *each of* $L, \mathcal{F}_1, R, \mathcal{F}_2$ *are* abecedarian *with respect to the same bucketing system as* $\mathcal{F}_0$

– *when* $a = b$, $f_{\mathsf{left}} = f_{\mathsf{left}}[a, a)$ $f_1 = f_1[a, a)$ $f_{\mathsf{right}} = f_{\mathsf{right}}[a, a)$;

– *when* $a < b$, *there exist* $a \le i \le j \le b$ *such that*

$$a = i < j = b \implies f_{\mathsf{left}} = f_{\mathsf{left}}[a, i) \quad f_1 = f_1[i, j) \quad f_{\mathsf{right}} = f_{\mathsf{right}}[j, b).$$
$$a = i = j < b \implies f_{\mathsf{left}} = f_{\mathsf{left}}[a, i) \quad f_1 = f_1[i, j) \quad f_{\mathsf{right}} = f_{\mathsf{right}}[j, b).$$
$$a = i < j < b \implies f_{\mathsf{left}} = f_{\mathsf{left}}[a, i) \quad f_1 = f_1[i, j{+}1) \quad f_{\mathsf{right}} = f_{\mathsf{right}}[j, b).$$
$$a < i = j = b \implies f_{\mathsf{left}} = f_{\mathsf{left}}[a, i{+}1) \quad f_1 = f_1[i, j) \quad f_{\mathsf{right}} = f_{\mathsf{right}}[j, b).$$
$$a < i = j < b \implies f_{\mathsf{left}} = f_{\mathsf{left}}[a, i{+}1) \quad f_1 = f_1[i{+}1, j{+}1) \quad f_{\mathsf{right}} = f_{\mathsf{right}}[j, b).$$
$$a < i < j = b \implies f_{\mathsf{left}} = f_{\mathsf{left}}[a, i{+}1) \quad f_1 = f_1[i, j) \quad f_{\mathsf{right}} = f_{\mathsf{right}}[j, b).$$
$$a < i < j < b \implies f_{\mathsf{left}} = f_{\mathsf{left}}[a, i+1) \quad f_1 = f_1[i, j+1) \quad f_{\mathsf{right}} = f_{\mathsf{right}}[j, b).$$

Before proving Claim 21, let us complete the proof of Lemma 6 using it.

By the above claim, we have a formula $\mathcal{F}_0'$ computing f_0 that looks like $L \cdot \mathcal{F}_1 \cdot R + \mathcal{F}_2$ where each of $L, \mathcal{F}_1, R, \mathcal{F}_2$ have size at most $(2s/3)$. Further if $\mathcal{F}$ is homogeneous, then so are each of $L, \mathcal{F}_1, R, \mathcal{F}_2$. Hence, $\mathcal{F}_0'$ is homogeneous. On the other hand, when $\mathcal{F}_0$ is $\mathsf{abecedarian}$, so are $L, \mathcal{F}_1, R, \mathcal{F}_2$. Further, note that $\mathcal{F}_0'$ is also $\mathsf{abecedarian}$ in this case since $f_{\mathsf{left}}, f_1, f_{\mathsf{right}}, f_2$ are of the *correct type* due to Claim 21.

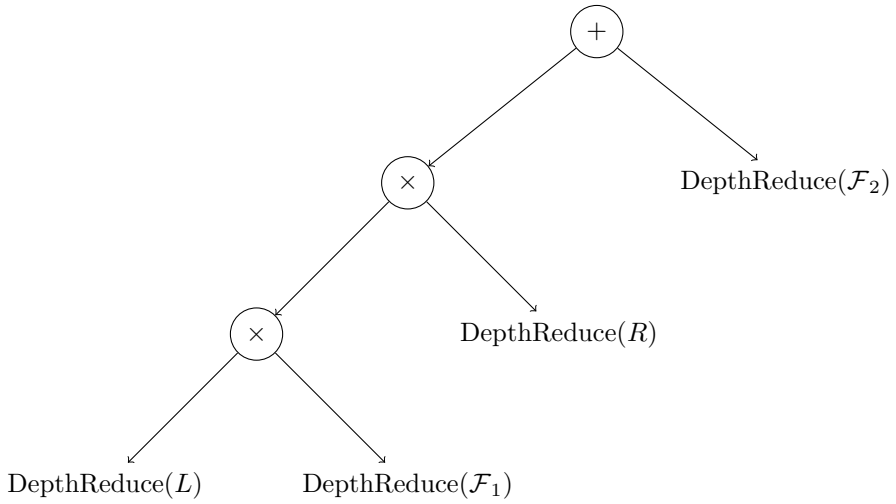
In all the cases, recursively applying this technique, on each of $L, \mathcal{F}_1, R, \mathcal{F}_2$, we get

$$\mathsf{depth}(s) \le \mathsf{depth}(2s/3) + 3 \qquad \text{and} \qquad \mathsf{size}(s) \le 4 \cdot \mathsf{size}(2s/3) + 3.$$

Note that in the base case, when s is constant, both $\mathsf{size}(s)$ and $\mathsf{depth}(s)$ are constants. Thus,

$$\mathsf{depth}(s) = O(\log s) \qquad \text{and} \qquad \mathsf{size}(s) = \mathrm{poly}(s). \qquad \square$$

Pictorially, once we have Claim 21, we essentially do the following recursively.



We now complete the proof of Claim 21

Proof of Claim 21. From the root let us traverse $\mathcal{F}_0$ towards the leaves, always choosing the child that has a larger sub-tree under it, till we find a vertex v such

that the associated sub-tree has size at most $(2s/3)$. Since $\mathcal{F}_0$ tree has fan-in 2, we also know that the size of this sub-tree must be at least $(s/3)$. Let this sub-tree be $\mathcal{F}_1$. Additionally, in the case when $\mathcal{F}_0$ is abecedarian, let us assume that v is labelled with $[i_v, j_v)$.

Let $\mathcal{P}$ be the path from v to the root and v_{add} the addition gate on $\mathcal{P}$ which is closest to v. Also let the set of multiplication gates on $\mathcal{P}$ be $\{v_1, \ldots, v_\ell\}$ for some $\ell \in \mathbb{N}$. Assume, without loss of generality, that v_1 is closest to v and v_ℓ to the root. Further, for every $i \in [\ell]$, let L_i be sub-formula corresponding to the left child of v_i and R_i the one to its right child. Note that for every $i \in [\ell]$, exactly one of children of v_i is a vertex in $\mathcal{P}$. We can then define L and R as follows.

Step 1: Set $L = R = 1$.

Step 2: For i from 1 to ℓ,

$$L = \begin{cases} L_i \times L & \text{if the right child of } v_i \text{ is a vertex in } \mathcal{P}, \\ L & \text{otherwise.} \end{cases}$$

and

$$R = \begin{cases} R & \text{if the right child of } v_i \text{ is a vertex in } \mathcal{P}, \\ R \times R_i & \text{otherwise.} \end{cases}$$

Also define $\mathcal{F}_2$ to be the formula we get by replacing the vertex v and the sub-tree under it with 0, and then removing the redundant gates.

Clearly, by construction, $\mathcal{F}_1$, L, R and $\mathcal{F}_2$ are sub-formulas of $\mathcal{F}_0$. Further, $\mathcal{F}_1$ is disjoint from L, R and $\mathcal{F}_2$. As a result, since $\mathcal{F}_1$ has size at least $(s/3)$ and at most $(2s/3)$, it must be the case that each of L, R and $\mathcal{F}_2$ have size at least $(s/3)$ and at most $(2s/3)$.

Also, it is not hard to see that $\mathcal{F}_0' = L \cdot \mathcal{F}_1 \cdot R + \mathcal{F}_2$ computes f_0. What is left to check is that when $\mathcal{F}_0$ is homogeneous or abecedarian, then $L, \mathcal{F}_1, R, \mathcal{F}_2$ have the additional properties claimed. The one line proof of this is that each *parse-tree*[6] of $\mathcal{F}_0$ is merely restructured in the above process, without changing its value. We however go over the proof explicitly for the sake of completeness.

When $\mathcal{F}_0$ is homogeneous, since $L, \mathcal{F}_1, R, \mathcal{F}_2$ are sub-formulas, they are also homogeneous.
On the other hand, suppose $\mathcal{F}_0$ is abecedarian and $f_0 = f_0[a, b)$. Recall that the vertex v was labelled by $[i_v, j_v)$. Let us set $i = i_v$ and $j = j_v$. Then, by definition, $\mathcal{F}_1$ is labelled by $[i, j)$. Hence, if f_1 is the polynomial computed at v, then $f_1 = f_1[i, j)$. Further, $\mathcal{F}_1$ is abecedarian since it is a sub-formula of $\mathcal{F}_0$ and computes an abecedarian polynomial.

Now let us focus on $\mathcal{F}_2$. Essentially $\mathcal{F}_2$ is got by removing from $\mathcal{F}_0$, v and all the multiplication gates on P between v and v_{add} along with the sub-trees under them. Thus $\mathcal{F}_2$ is also abecedarian in this case, and if f_2 is the polynomial by it, then $f_2 = f_2[a, b)$.

Finally, note that the left indices of labels on the various vertices of $\mathcal{P}$ change only at the gates at which multiplications to L occur. Further, note that they occur in the *correct order* and are of the *correct type*. Thus, by induction, it is easy to see that the labels on L are consistent with those on the L_is

[6] For a definition, see for example [LLS19]

when the respective multiplications happen. Therefore L is $\mathsf{abecedarian}$, and $f_{\mathsf{left}} = f_{\mathsf{left}}[a, i)$.

For similar reasons, R is also $\mathsf{abecedarian}$ and $f_{\mathsf{right}} = f_{\mathsf{right}}[j, b)$. This completes the proof. □

3.2 Homogenisation

Raz [Raz13] had shown that if there is a formula computing a homogeneous polynomial of *low* degree in the commutative world, then it can be assumed without loss of generality that the formula is homogeneous. We show that his proof also works in the non-commutative setting because of Lemma 6. A complete proof is given here for the sake of completeness.

Lemma 5 (Homogenising Non-Commutative Formulas computing Low Degree Polynomials)**.** *Suppose f is a non-commutative homogeneous polynomial of degree $d = O(\log s)$, where s is its ABP complexity. Also, suppose $\mathcal{F}$ is a fan-in 2 formula of size s' that computes f. Then there is a homogeneous formula $\mathcal{F}'$ computing f, that has size* $\mathrm{poly}(s')$ *and whose multiplication gates have fan-in 2.*

Further, if $\mathcal{F}$ was $\mathsf{abecedarian}$ *with respect to some bucketing system, then $\mathcal{F}'$ is also* $\mathsf{abecedarian}$ *with respect to the same bucketing system.*

Proof. We first note that since s is the ABP complexity of f, $s' \geq s$. Further if $\mathcal{F}$ has depth r, then by Lemma 6, we can assume without loss of generality, that $r = O(\log s')$.

In order to construct a homogeneous formula computing f, we first homogenise $\mathcal{F}$ to obtain a circuit $\mathcal{C}$, and then *unravel* $\mathcal{C}$ to make it into a formula $\mathcal{F}'$.

The first step is done in the usual manner. For every gate v in $\mathcal{F}$, we have $d+1$ gates $(v, 0), \ldots, (v, d)$ in $\mathcal{C}$. Intuitively if f_v is the polynomial computed at v, then the polynomial computed at (v, i) is the degree i homogeneous component of f_v. These vertices are then connected as follows.

- If $v = u_1 + u_2$, then for every $i \in \{0, \ldots, d\}$, $\quad (v, i) = (u_1, i) + (u_2, i)$.
- If $v = u_1 \times u_2$, then for every $i \in \{0, \ldots, d\}$, $\quad (v, i) = \sum_{j=0}^{i} (u_1, j) \times (u_2, i - j)$.

So, we now have a homogeneous circuit $\mathcal{C}$ that computes f and has size at most $O(d^2 \cdot s')$. Also, the depth of this circuit is at most twice that of $\mathcal{F}$, and the multiplication gates have fan-in 2.

To convert $\mathcal{C}$ into a formula $\mathcal{F}'$, we have to recompute nodes whenever they have to be reused. That is, a particular vertex in $\mathcal{C}$ has to be duplicated as many times as there are paths from the vertex to the root. Thus, to upper bound the size of $\mathcal{F}'$, we need to give an upper bound on the number of distinct paths from every vertex of $\mathcal{C}$ to its root.

Let us arbitrarily choose a vertex (v, i) in $\mathcal{C}$, and consider the path from it to the root. Suppose the path is $(v, i) = (v_1, i_1) \to \cdots \to (v_\ell, i_\ell) = (\mathsf{root}, d)$ where ℓ is at most the depth of $\mathcal{C}$. Note that it must be the case that $i = i_1 \leq \cdots \leq i_\ell = d$. Hence, if we define $\delta_j = i_{j+1} - i_j$ for $j \in [\ell - 1]$, then the δ_js are non-negative

integers such that $\delta_1 + \cdots + \delta_{\ell-1} = (d-i)$. Thus, the number of choices we have for $(i_2, \ldots, i_\ell)$ such that $i = i_1 \leq \cdots \leq i_\ell = d$, is the same as the number of choices we have for $(\delta_1, \ldots, \delta_{\ell-1})$ such that $\delta_1 + \cdots + \delta_{\ell-1} = (d-i) \leq d$. This is at most $\binom{\ell+d}{\ell}$.

Note that in this process the fan-in of the gates have not changed, and hence the multiplication gates in $\mathcal{F}'$ continue to have fan-in 2. Further, we know that the $\mathcal{C}$ has depth $2r$ and hence $\ell \leq 2r$. Therefore, the number of paths from (v, i) to the root is at most $\binom{2r+d}{2r}$. Hence, if $\mathcal{F}'$ is the formula obtained by unravelling $\mathcal{C}$, then $\mathsf{size}(\mathcal{F}') \leq s' \cdot d^2 \cdot \binom{2r+d}{d}$. Here $r = O(\log(s'))$, and $s \leq s'$ implying that $d = O(\log(s)) = O(\log(s'))$. Thus, $\mathsf{size}(\mathcal{F}') \leq \mathrm{poly}(s')$.

Finally, assume that $\mathcal{F}$ is abecedarian. Then every vertex v is labelled with a tuple of bucket indices, say (a_v, b_v). In that case, we add the label (a_v, b_v) to the gates $\{(v, i)\}_{i=0}^{d}$ in $\mathcal{C}$ and continue with the proof as is. Note that the final formula that we get, $\mathcal{F}'$, is abecedarian and all the other properties that were true in the general case, continue to be true. □

4 Converting Computational Models into Abecedarian Ones

In this section we show that, without loss of generality, circuits and ABPs computing abecedarian polynomials can be assumed to be abecedarian. For formulas however, we can prove such a statement only in certain cases.

4.1 Circuits

Hrubeš *et al.* [HWY11] had shown that any circuit computing an ordered polynomial can be assumed to be ordered without loss of generality.

Theorem 22 (Theorem 7.1 in [HWY11])**.** *Let $\mathcal{C}$ be a circuit of size s computing an ordered polynomial f of degree d. Then, there is an ordered circuit $\mathcal{C}'$ of size $O(d^3 s)$ that computes f.* □

We show that the proof of this statement can be generalised to show Observation 7. A complete proof is given for the sake of completeness.

Observation 7 (Converting Circuits into Abecedarian Circuits)**.** *Let f be an* abecedarian *polynomial with respect to a bucketing system of size m, and $\mathcal{C}$ be a circuit of size s computing f. Then there is an* abecedarian *circuit $\mathcal{C}'$ computing f of size $O(m^3 s)$.*

Proof. Without loss of generality, let us assume that $\mathcal{C}$ has fan-in 2.

We prove the given statement by describing how to construct $\mathcal{C}'$ from $\mathcal{C}$. For each gate v in $\mathcal{C}$, we make $O(m^2)$ copies in $\mathcal{C}'$, $\{(v, [a, b)) \ : \ 1 \leq a \leq b \leq m+1\}$; and if root is the output gate in $\mathcal{C}$, then we define the set of output gates in $\mathcal{C}'$ to be $\{(\mathsf{root}, [i, m+1))\}_{i \in [m+1]}$.

Intuitively, if f_v is the polynomial computed at v in $\mathcal{C}$, then the polynomial computed at $(v, [a, b))$ is $f_v[a, b)$. Thus if f was the polynomial computed at root, then the polynomial computed by $\mathcal{C}'$ is $\sum_{i=1}^{m+1} f[i, m+1)$ which is indeed f.

We ensure this property at every gate by adding edges as follows.

- If v is an input gate labelled by a field element γ,
  - we set $(v, [a, a)) = \gamma$ for every $a \in [m+1]$;
  - we set $(v, [a, b)) = 0$ for every $1 \leq a < b \leq m+1$.
- If v is an input gate labelled by a variable x_i and $x_i \in X_k$,
  - we set $(v, [k, k+1)) = x_i$;
  - we set $(v, [a, b)) = 0$ for every $a \neq k$, $b \neq k+1$.
- If $v = v_1 + v_2$, we set $(v, [a, b)) = (v_1, [a, b)) + (v_2, [a, b))$ for every $a \leq b \in [m+1]$.
- If $v = v_1 \times v_2$, we set $(v, [a, a)) = (v_1, [a, a)) \cdot (v_2, [a, a))$ for every $a \in [m+1]$; and

$$(v, [a, b)) = (v_1, [a, a)) \cdot (v_2, [a, b)) + (v_1, [a, b)) \cdot (v_2, [b, b)) + \sum_{c=a}^{b-1} (v_1, [a, c+1)) \times (v_2, [c, b))$$

  for every $1 \leq a < b \leq m+1$.

Finally, for every $1 \leq a \leq b \leq m+1$, we associate the gate $(v, [a, b))$ in $\mathcal{C}'$ with the set $[a, b)$.

Using induction, one can easily show that the gates in $\mathcal{C}'$ have the claimed properties. Hence $\mathcal{C}'$ is indeed an abecedarian circuit computing f. Further for every gate v in $\mathcal{C}$, there are at most $O(m^3)$ vertices in $\mathcal{C}'$. Thus the size of $\mathcal{C}'$ is $O(m^3 s)$. □

4.2 Algebraic Branching Programs

Next, we show that a similar statement is true for ABPs as well.

Observation 8 (Converting ABPs into Abecedarian ABPs)**.** *Suppose f is an* abecedarian *polynomial with respect to a bucketing system of size m. If there is an ABP $\mathcal{A}$ of size s computing it, then there is an* abecedarian *ABP $\mathcal{A}'$ computing it of size $O(ms)$.*

Proof. Let f have degree d and be abecedarian with respect to the buckets $\{X_i\}_{i=1}^m$, where $X_i = \{x_{i,j} : j \in [n_i]\}$. Without loss of generality, we can assume that $\mathcal{A}$ is homogeneous[7]. If f is not homogeneous, $\mathcal{A}$ can be thought of as a collection of homogeneous ABPs $\{\mathcal{A}_1, \ldots, \mathcal{A}_d\}$ where $\mathcal{A}_k$ computes the k-th homogeneous component of f.

We prove the theorem by describing how to construct $\mathcal{A}'$. For each vertex v in $\mathcal{A}$, make $O(m)$ copies in $\mathcal{A}'$, namely $\{(v, a) : 0 \leq a \leq m\}$. Intuitively, if $g_{(u,v)}$ is the polynomial computed between u and v in $\mathcal{A}$, then the polynomial computed between (u, a) and (v, b) in $\mathcal{A}'$ is $g_{(u,v)}[a, b+1)$. The way we ensure this property at every vertex is by adding edges in $\mathcal{A}'$ as follows.

> For any two vertices u, v in $\mathcal{A}$, suppose there is an edge between them that is labelled with $\sum_{i \in [m]} \sum_{j \in [n_i]} \gamma_{i,j} x_{i,j}$. Then, for every $a, b \in [m]$ with $a \leq b$, add an edge from (u, a) to (v, b) with label $\sum_{i=a}^{b} \left(\sum_{j \in [n_i]} \gamma_{i,j} x_{i,j} \right)$.

[7] Every edge is labelled by a homogeneous form.

Also, associate the bucket index a with the gate (v, a) in $\mathcal{A}'$.

By induction, one can easily show that the gates in $\mathcal{A}'$ have the claimed property. Hence $\mathcal{A}'$ is indeed an abecedarian ABP computing f. Further, every vertex v in $\mathcal{A}$, there are at most $O(m)$ vertices in $\mathcal{A}'$. Therefore, the size of $\mathcal{A}'$ is $O(ms)$. □

4.3 Formulas

Finally we show that in the case of formulas, we can prove a similar statement only when the polynomial is abecedarian with respect to a bucketing system of *small size*. The proof is very similar to that of Lemma 5.

Theorem 2 (Converting Formulas into Abecedarian Formulas)**.** *Let f be an* abecedarian *polynomial with respect to a bucketing system of size m, and $\mathcal{F}$ be a formula of size s computing f. If $m = O(\log s)$, then there is an* abecedarian *formula $\mathcal{F}'$ computing f of size* $\mathrm{poly}(s)$.

Proof. Let us assume additionally that $\mathcal{F}$ has depth r. Now Lemma 6 implies that $r = \log(s)$ without loss of generality. By Observation 7, there is an abecedarian circuit $\mathcal{C}$ that computes f and has size at most $s' = O(s \cdot m^3)$. Further its proof implies that the depth of $\mathcal{C}$ is at most $2r$.

To convert $\mathcal{C}$ into an abecedarian formula $\mathcal{F}'$, we have to recompute a node each time it has to be reused. That is, a particular vertex in $\mathcal{C}$ has to be duplicated as many times as there are paths from the vertex to the root. Thus to upper bound the size of $\mathcal{F}'$, we need to give an upper bound on the number of distinct paths from every vertex in $\mathcal{C}$ to its root.

Let us arbitrarily choose a vertex $(v, [a, b))$ in $\mathcal{C}$, and consider the path from it to the root. Suppose the path is $(v, [a, b)) = (v_1, [a_1, b_1)) \to \cdots \to (v_\ell, [a_\ell, b_\ell)) = (\mathsf{root}, [i, m+1))$ for some ℓ that is at most the depth of $\mathcal{C}$. Note that it must be the case that

$$i \le a_\ell \le \cdots \le a_1 \le a \le b \le b_1 \le b_\ell \le m + 1.$$

Let us define $\delta_j = a_j - a_{j+1}$ and $\delta'_j = b_{j+1} - b_j$ for $j \in [\ell - 1]$. Then, the number of choices we have for $(a_1, \ldots, a_\ell)$ and $(b_1, \ldots, b_\ell)$ such that

$$i = a_\ell \le \cdots a_1 = a \le b = b_1 \le \cdots \le b_\ell = m + 1$$

is the same as the number of choices we have for $(\delta_1, \ldots, \delta_{\ell-1}, \delta'_1, \ldots, \delta'_{\ell-1})$ such that

$$\delta_1 + \cdots + \delta_{\ell-1} + \delta'_1 + \cdots + \delta'_{\ell-1} = (m + 1 - (b - a) - i) \le m.$$

This is clearly at most $\binom{2\ell+m}{m}$.

Further, we know that the $\mathcal{C}$ has depth $2r$ and hence $\ell \le 2r$. Therefore, the number of paths from (v, i) to the root is at most $\binom{4r+m}{m}$. Hence if $\mathcal{F}'$ is the formula obtained by unravelling $\mathcal{C}$, then $\mathsf{size}(\mathcal{F}') \le s' \cdot m^2 \cdot \binom{4r+m}{m}$. Here $s' = O(m^3 \cdot s)$, $r = O(\log(s))$ and $m = O(\log(s))$. Thus, $\mathsf{size}(\mathcal{F}') \le \mathrm{poly}(s)$. □

References

[AJR16] V. Arvind, P. S. Joglekar, and S. Raja. Noncommutative Valiant's Classes: Structure and Complete Problems. *ACM Trans. Comput. Theory*, 9(1), October 2016.

[AS18] Vikraman Arvind and Srikanth Srinivasan. On the hardness of the noncommutative determinant. *Comput. Complex.*, 27(1):1–29, 2018.

[BC92] Michael Ben-Or and Richard Cleve. Computing Algebraic Formulas Using a Constant Number of Registers. *SIAM Journal of Computing*, 21(1):54–58, 1992.

[Bre74] Richard P. Brent. The Parallel Evaluation of General Arithmetic Expressions. *Journal of the ACM*, 21(2):201–206, April 1974.

[CILM18] Marco L. Carmosino, Russell Impagliazzo, Shachar Lovett, and Ivan Mihajlin. Hardness Amplification for Non-Commutative Arithmetic Circuits. In Rocco A. Servedio, editor, *Computational Complexity Conference (2018)*, volume 102 of *LIPIcs*, pages 12:1–12:16. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2018. ECCC TR18-095.

[CKSV19] Prerona Chatterjee, Mrinal Kumar, Adrian She, and Ben Lee Volk. A Quadratic Lower Bound for Algebraic Branching Programs and Formulas. *CoRR*, 1911.11793v2, 2019. Pre-print available at `arXiv:1911.11793v2`.

[DMPY12] Zeev Dvir, Guillaume Malod, Sylvain Perifel, and Amir Yehudayoff. Separating multilinear branching programs and formulas. In Howard J. Karloff and Toniann Pitassi, editors, *Proceedings of the 44th Symposium on Theory of Computing Conference, STOC 2012, New York, NY, USA, May 19 - 22, 2012*, pages 615–624. ACM, 2012.

[FLOS20] Nathanaël Fijalkow, Guillaume Lagarde, Pierre Ohlmann, and Olivier Serre. Lower Bounds for Arithmetic Circuits via the Hankel Matrix. In *Symposium on Theoretical Aspects of Computer Science (2020)*, volume 154, pages 24:1–24:17, Montpellier, France, March 2020.

[HWY10] Pavel Hrubes, Avi Wigderson, and Amir Yehudayoff. Relationless Completeness and Separations. In *Proceedings of the 25th Annual IEEE Conference on Computational Complexity, CCC 2010, Cambridge, Massachusetts, USA, June 9-12, 2010*, pages 280–290. IEEE Computer Society, 2010.

[HWY11] Pavel Hrubeš, Avi Wigderson, and Amir Yehudayoff. Noncommutative circuits and the sum-of-squares problem. *Journal of the American Mathematical Society*, 24(3):871–898, 2011.

[HY11] Pavel Hrubes and Amir Yehudayoff. Homogeneous Formulas and Symmetric Polynomials. *Comput. Complex.*, 20(3):559–578, 2011.

[Hya77] L. Hyafil. The power of commutativity. In *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, pages 171–174, 1977.

[Kal85] Kyriakos Kalorkoti. A Lower Bound for the Formula Size of Rational Functions. *SIAM Journal of Computing*, 14(3):678–687, 1985.

[LLS19] Guillaume Lagarde, Nutan Limaye, and Srikanth Srinivasan. Lower Bounds and PIT for Non-commutative Arithmetic Circuits with Restricted Parse Trees. *Comput. Complex.*, 28(3):471–542, 2019.

[LMP19] Guillaume Lagarde, Guillaume Malod, and Sylvain Perifel. Non-commutative computations: lower bounds and polynomial identity testing. *Chic. J. Theor. Comput. Sci.*, 2019, 2019.

[LMS16] Nutan Limaye, Guillaume Malod, and Srikanth Srinivasan. Lower Bounds for Non-Commutative Skew Circuits. *Theory of Computing*, 12(1):1–38, 2016. ECCC TR15-22.

[MW19] Merriam and Webster. Definition of Abecedarian. Word of the Day at www.merriam-webster.com, 2019.

[Nec66] Eduard Ivanovich Nechiporuk. On a Boolean function. *Dokl. Akad. Nauk SSSR*, 169:765–766, 1966.

[Nis91] Noam Nisan. Lower bounds for non-commutative computation. In *ACM Symposium on Theory of Computing (1991)*, pages 410–418, 1991. Available on `citeseer:10.1.1.17.5067`.

[Raz13] Ran Raz. Tensor-Rank and Lower Bounds for Arithmetic Formulas. *J. ACM*, 60(6):40:1–40:15, 2013.

[Rys63] Herbert J. Ryser. Combinatorial Mathematics. *Mathematical Association of America*, 14, 1963.

[ST18] Ramprasad Saptharishi and Anamay Tengse. Quasipolynomial Hitting Sets for Circuits with Restricted Parse Trees. In *Foundations of Software Technology and Theoretical Computer Science (2018)*, pages 6:1–6:19, 2018.

[Val79] Leslie G. Valiant. Completeness Classes in Algebra. In *ACM Symposium on Theory of Computing (1979)*, pages 249–261, 1979.

[VSBR83] Leslie G. Valiant, Sven Skyum, S. Berkowitz, and Charles Rackoff. Fast Parallel Computation of Polynomials Using Few Processors. *SIAM Journal of Computing*, 12(4):641–644, 1983. Preliminary version in Mathematical Foundations of Computer Science (1981).